

Demos 시스템 배포 및 로컬 환경 구축 가이드 v0.2

1. 서론: Demos 시스템의 전략적 가치 및 개요

Demos(데모스) 프로젝트는 단순한 인터페이스를 넘어, 10개 전문 분야의 174개 과학적 스킬셋을 LLM에 동적으로 주입하는 지능형 과학 연구 어시스턴트입니다. 본 시스템은 조직 내 보안 가이드라인을 준수하는 동시에, 연구 데이터의 외부 유출 없이 고성능 추론 능력을 확보하도록 설계되었습니다. Demos 시스템은 인프라 자원의 가용성과 보안 요구 수준에 따라 다음과 같은 4가지 계층적 LLM 환경을 전략적으로 운영합니다.

-  **DEV (30B):** Qwen3-Coder-30B-A3B-Instruct 모델 기반. 개발 및 샌드박스 테스트를 위한 안정적인 실험 환경을 제공합니다.
-  **PROD (80B):** Qwen3-Next-80B-A3B-Instruct 모델 기반. 최고 수준의 추론 성능이 요구되는 실무 및 전문 연구용 운영 서버입니다.
-  **COMMON (20B):** gpt-oss-20b 모델 기반. 일반적인 질의와 빠른 응답 속도가 필요한 범용 업무에 최적화되어 있습니다.
-  **LOCAL GGUF:** local-gguf 모델 기반. 외부 네트워크와 완전히 차단된 환경에서 로컬 GPU/CPU 리소스를 활용하는 독립형 추론 환경입니다. 이러한 다각적 아키텍처는 연구자가 작업의 복잡도와 보안 등급에 맞춰 최적의 모델을 선택하게 함으로써 연구 생산성을 극대화합니다. 이제 시스템의 안정적인 배포를 위한 기술적 토대인 필수 요구사항을 확인하겠습니다.

2. 인프라 구축을 위한 사전 필수 요구사항

시스템 운영의 무결성을 보장하기 위해, 배포 전 소프트웨어 및 네트워크 환경이 아래 명시된 기술 요건을 충족해야 합니다.

소프트웨어 및 네트워크 기술 사양

- **Python 3.8+ Runtime:** 현대적인 비동기 처리 및 라이브러리 호환성을 위한 최소 엔진 사양입니다.
- **핵심 의존성:** 웹 프레임워크인 **Flask** 와 API 통신을 위한 **requests** 패키지가 필수적입니다.
- **포트 점유 정책:** 시스템은 기본적으로 **10009** 포트를 점유합니다. 배포 전 **netstat** 등을 통해 해당 포트의 가용 여부를 반드시 진단하십시오.
- **운영 타임아웃(120s):** 분자 설계, 단백질 구조 예측, 복잡한 화학 시뮬레이션 등 174개 과학 스킬 기반의 고부하 쿼리는 일반적인 질의보다 긴 처리 시간을 요구합니다. 시스템은 지연으로 인한 세션 단절을 방지하기 위해 **120초**의 관대한 타임아웃 정책을 적용합니다. 위 요구사항이 충족되었다면, 보안 환경에서의 가장 큰 허들인 패키지 배포 프로세스를 진행합니다.

3. 폐쇄망 환경을 위한 패키지 배포 전략

엄격한 보안 정책이 적용된 연구 환경에서는 외부 인터넷 연결이 제한됩니다. Demos 시스템은 '오프라인 배포 모델'을 통해 시스템 무결성을 유지하면서도 신속한 구축을 지원합니다.

2단계 오프라인 설치 메커니즘

1. **Stage 1: 외부망(Staging Zone):** 인터넷 통신이 가능한 PC에서 시스템 구동에 필요한 모든 종속성 라이브러리를 사전에 패키징합니다.

2. **Stage 2: 내부망(Deployment Zone):** 확보된 패키지 파일을 내부망 서버로 안전하게 이관한 후, 외부 저장소(PyPI)를 참조하지 않는 **--no-index** 모드로 설치를 수행합니다. 패키지 설치가 완료되면, 실제 서비스 연동을 위한 인증 및 엔드포인트 최적화 단계로 진입합니다.

4. 시스템 인증 및 API 엔드포인트 최적화 설정

내부 API 호출의 안정성을 확보하기 위해 **TOKEN.TXT**를 통한 인증 체계와 엔드포인트 관리를 구성해야 합니다.

API 인증 및 연결 최적화 가이드

- 인증 파일(**TOKEN.TXT**): **app.py**와 동일한 루트 경로에 위치해야 하며, 발급받은 API 키를 단일 행으로 기록합니다.
- **SSL 검증 전략:** 폐쇄망 내에서 자체 서명 인증서(**Self-signed Certificate**) 문제로 인한 통신 장애를 방지하기 위해 **verify=False** 설정을 적용합니다. 이는 내부망 환경에서 보안 정책과 운영의 연속성 사이의 균형을 맞추기 위한 전략적 트레이드오프입니다.
- 엔드포인트 최적화: **app.py** 내 **ENV_CONFIG** 수정을 통해 조직 내 각 모델 서버(**DEV/PROD/COMMON**)의 주소를 환경에 맞게 맵핑할 수 있습니다. **!CAUTION TOKEN.TXT** 작성 준수 원칙
- **Character Set:** 반드시 **ASCII**(영문 및 숫자) 문자열만 사용해야 합니다.
- 인코딩 오류 방지: 파일 내에 한글이나 특수문자가 포함될 경우 **latin-1** 인코딩 에러가 발생하여 인증 시스템이 중단됩니다.
- 예외 케이스: 오직 **LOCAL GGUF** 모드만 단독 운영할 경우, 본 파일은 생성하지 않거나 비워두어도 무방합니다. 서버 기반 API 설정이 완료되었다면, 시스템의 독립적 운영 가능성을 부여하는 로컬 추론 환경 구축을 살펴보겠습니다.

5. LOCAL GGUF 모델 연동 및 자동 감지 메커니즘

Demos 시스템은 로컬 하드웨어 리소스를 활용하는 **GGUF** 추론 엔진(**llama-server**)을 내장 관리하며, 이를 통해 완전한 오프라인 환경을 제공합니다.

자동화된 추론 엔진 로직

시스템 구동 시 다음 두 가지 조건이 모두 충족되면 **LOCAL GGUF** 환경이 자동으로 활성화 됩니다.

1. 모델 무결성: **.gguf** 파일이 **app.py** 실행 폴더, **models/**, 혹은 **model/** 디렉토리 내에 존재해야 합니다.
2. 엔진 가용성: **llama-server** 실행 파일이 **app.py** 폴더, **llama.cpp/build/bin/** 경로 내에 있거나 시스템 **PATH**에 등록되어 있어야 합니다. 아키텍트 관점의 자원 관리: 메인 애플리케이션이 **SIGINT(Ctrl+C)** 신호를 수신하여 종료될 때, 백그라운드에서 구동 중인 **llama-server** 프로세스도 자동으로 함께 종료 되도록 설계되어 좀비 프로세스 발생을 원천 차단합니다.

6. 과학 스킬(Scientific Skills) 구조화 및 커스터마이징

Demos의 핵심 가치는 10개 분야, 174개에 달하는 전문 과학 스킬셋에 있습니다. 각 스킬은 **SKILL.md** 파일 형태로 관리되며, 사용자의 선택에 따라 **LLM**의 시스템 프롬프트로 주입됩니다.

10대 주요 분야 및 스킬 구성

분야,스킬 수,대표 스킬 예시

-  생물정보학,22개,"biopython, scanpy, esm"
-  생물 DB,22개,"uniprot, KEGG, AlphaFold"
-  화학/신약,21개,"rdkit, deepchem, diffdock"
-  재료/물리/양자,8개,"pymatgen, qiskit, circ"
-  데이터/ML,26개,"scikit-learn, pytorch, polars"
-  금융/경제,8개,"FRED, edgar, USPTO"
-  임상/의학,13개,"pydicom, pathml, ClinicalTrials"
-  논문/연구,20개,"scientific-writing, PubMed"
-  랩 자동화,12개,"opentrons, benchling, lamindb"
-  유틸리티,21개,"docx, xlsx, pdf, pptx"

이 시스템은 동적 로딩(**Dynamic Loading**) 구조를 채택하여, **scientific-skills/** 디렉토리에 새로운 스킬 폴더와 **SKILL.md**를 추가하는 것만으로 서버 재시작 없이 즉각적인 기능 확장이 가능합니다.

7. 기술적 트러블슈팅 및 운영 최적화

초기 배포 및 운영 단계에서 발생할 수 있는 주요 병목 현상과 해결 방안을 기술합니다.| 증상 (Symptom) | 발생 원인 (Cause) | 해결책 (Solution) || ----- | ----- | ----- || **latin-1** 인코딩 에러 | TOKEN.TXT에 비ASCII 문자(한글 등) 포함 | 영문/숫자로만 구성된 API 키로 파일 재작성 || **GGUF** 버튼 미활성화 | llama-server 엔진 또는 모델 파일 경로 오류 | app.py 폴더 혹은 llama.cpp/build/bin/ 확인 || **API** 연결 실패 | 엔드포인트 오기입 또는 네트워크 방화벽 차단 | ENV_CONFIG 내 URL 검증 및 10009 포트 허용 || 스킬 선택 시 **✗** 표시 | 해당 스킬 디렉토리 내 **SKILL.md** 누락 | scientific-skills/폴더명/SKILL.md 파일 생성 || 포트 충돌 현상 | 10009 포트가 타 서비스에 의해 점유됨 | app.py 하단의 port=10009를 미점유 포트로 수정 |

8. 결론: 안정적 운영을 위한 체크리스트 및 향후 전망

Demos 시스템은 폐쇄망 환경의 제약을 극복하고 연구자에게 최첨단 **AI** 자원을 제공하는 유연한 인프라를 지향합니다. 본 가이드를 준수하여 구축된 환경은 시스템의 ****안정성(Stability)****과 ****확장성(Scalability)****을 동시에 보장하며, 향후 더 넓은 과학적 영역으로의 확장을 뒷받침할 것입니다.배포 완료 후, 운영 안정성 확보를 위해 다음 5가지 핵심 항목을 최종 점검하십시오.

1. 런타임 환경: Python 3.8+ 엔진 및 Flask/Requests 패키지 설치 여부.
2. 인증 무결성: TOKEN.TXT가 ASCII로 작성되어 app.py와 동일 경로에 존재하는가?
3. 로컬 엔진: llama-server와 .gguf 파일이 지정된 경로에 위치하여 자동 감지되는가?
4. 네트워크 가용성: 10009 포트 개방 및 http://localhost:10009 접속 확인.
5. 스킬 무결성: UI 상에서 각 분야별 스킬 카드에 ****✅ 표시(정상 로드)****가 나타나는가?이 가이드를 통해 구축된 **Demos** 시스템이 조직의 전략적 연구 자산으로 성공적으로 안착하기를 바랍니다.

제작:이준력